

Iterators and Generators in Python

A Conceptual Journey from Confusion to Internal Mastery

Mahesh Kumar. P

Contents

1	Iterators and Generators in Python	3
1.1	The Problem Before Iterators	3
1.1.1	Why Existing Solutions Fail	3
1.1.2	The Need	4
1.2	What Is Iteration?	4
1.2.1	Real-World Analogies	4
1.3	Iterable: The Object You Can Ask to Be Walked Through	5
1.3.1	Problem	5
1.3.2	Intuition	5
1.3.3	Examples	5
1.3.4	Why Integer Is Not Iterable	5
1.3.5	Memory Diagram	5
1.4	Iterator: The Object That Remembers the Walk	6
1.4.1	Problem	6
1.4.2	Inventing the Iterator Naturally	6
1.4.3	Manual Iteration	6
1.4.4	Line-by-Line Output Explanation	6
1.4.5	Iterator State Diagram	6
1.5	The Iteration Protocol	7
1.5.1	Problem	7
1.5.2	Need	7
1.5.3	Visualization	7
1.6	How a <code>for</code> Loop Works Internally	7
1.6.1	Problem	7
1.6.2	Internal Translation	8
1.6.3	Dry Run Table	8
1.6.4	Call Sequence Diagram	8
1.7	Custom Iterators	8
1.7.1	Why Build One?	8
1.7.2	Example 1: Counter Iterator	9
1.7.3	Example 2: Countdown Iterator	11
1.7.4	Example 3: Even Numbers Iterator	11
1.7.5	Example 4: Odd Numbers Iterator	12
1.7.6	Example 5: Alphabet Iterator	12
1.8	<code>StopIteration</code>	13
1.8.1	Why It Exists	13
1.8.2	What Happens If It Is Not Raised?	13
1.8.3	Why Users Rarely See It	13
1.9	Limitations of Custom Iterators	13
1.10	Introducing Generators	14

1.10.1	Problem	14
1.10.2	Curiosity	14
1.11	What Is a Generator?	14
1.11.1	Generator Function and Generator Object	14
1.12	Yield: Return Versus Pause-and-Resume	14
1.12.1	Real-World Analogy	15
1.12.2	Return vs Yield Table	15
1.12.3	Execution Flow	15
1.13	Generator Internal Execution	15
1.13.1	Complete Example	15
1.13.2	Console Output	16
1.13.3	Memory After Every Yield	16
1.14	Generator Expressions	16
1.14.1	Problem	16
1.14.2	Generator Expression	16
1.14.3	Benchmark Discussion	17
1.15	Lazy Evaluation	17
1.15.1	Why It Is Powerful	17
1.16	Infinite Generators	17
1.16.1	Safe Infinite Counter	17
1.16.2	Fibonacci Generator	18
1.16.3	Prime Number Generator	18
1.16.4	Streaming Sensor Analogy	18
1.17	Real-World Applications	19
1.17.1	Reading Huge Files	19
1.17.2	CSV Processing	19
1.17.3	Log Stream Processor	19
1.17.4	Database Pagination	19
1.17.5	Batch Processor	19
1.17.6	Data Pipeline	20
1.18	Iterator vs Generator	20
1.19	A Universal Code Example Checklist	20
1.20	Fifty Common Beginner Mistakes	21
1.21	Interview Questions and Model Answers	23
1.22	Practice Questions	23
1.23	Mini Projects	24
1.24	Summary	25
1.24.1	Cheat Sheet	25
1.24.2	Mind Map	25
1.24.3	Flow Chart	26
1.24.4	Revision Notes	26
1.24.5	Memory Tricks	26

Chapter 1

Iterators and Generators in Python

Chapter Promise

This chapter is not a syntax tour. It is a guided journey from the question “How does a `for` loop know what comes next?” to the ability to design custom iterators, write generators, reason about `yield`, explain memory behavior, and discuss the iteration protocol confidently in interviews.

1.1 The Problem Before Iterators

Start with ordinary code:

```
1 numbers = [10, 20, 30]
2
3 for item in numbers:
4     print(item)
```

It looks simple, but it hides several deep questions:

- How does Python know that 10 comes first?
- How does Python know that 20 comes next?
- Where is the current position stored?
- How does Python know the list is finished?
- Why can the same `for` loop work on lists, tuples, strings, dictionaries, sets, files, and ranges?

Curiosity Before Vocabulary

Do not begin by memorizing words such as *iterable*, *iterator*, or *protocol*. Begin with the mystery: Python needs a common way to ask many different objects, “Can I walk through you one value at a time?”

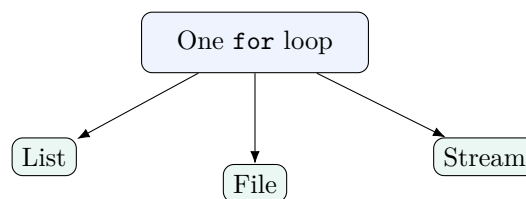
1.1.1 Why Existing Solutions Fail

A beginner might think Python simply uses indexes internally:

```
1 i = 0
2 while i < len(numbers):
3     print(numbers[i])
4     i += 1
```

That works for lists, tuples, strings, and ranges. But it fails as a universal idea:

- A dictionary is not normally processed by numeric indexes.
- A set has no stable index order.
- A file may be too large to index fully in memory.
- A network stream may not even have all future data yet.
- An infinite sequence has no final length.



A universal loop cannot depend on indexing, length, or storing all data at once.

1.1.2 The Need

Python needs a tiny contract:

1. Give me an object that knows how to produce values one at a time.
2. Let me ask that object for the next value.
3. Let that object tell me when it is finished.

That contract is the heart of iteration.

1.2 What Is Iteration?

Concept Before Syntax

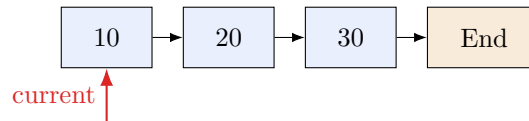
Iteration means repeatedly moving through a sequence, stream, collection, or process one item at a time while remembering where you currently are and knowing when there is no next item.

1.2.1 Real-World Analogies

Analogy	Current Position	Next / End Idea
Books on a shelf	Finger points to current book	Move to next book; stop after last shelf position
Reading pages	Bookmark remembers page	Turn page; stop at final page
Music playlist	Player remembers current song	Play next song; stop after queue ends
Customer queue	Clerk tracks front customer	Serve next customer; stop when line is empty
Factory conveyor	Sensor sees current package	Process next package; wait or stop when none exists

Every analogy has three ingredients:

- **Current position:** where the process is now.
- **Next item:** what should be produced next.
- **End signal:** how the process announces completion.



1.3 Iterable: The Object You Can Ask to Be Walked Through

1.3.1 Problem

We have many containers: lists, tuples, strings, dictionaries, sets, ranges, and files. Python needs to know which objects can participate in iteration.

1.3.2 Intuition

An **iterable** is an object that can provide a way to walk through its values one at a time. It is called iterable because it can be iterated over.

Iterable

An iterable is not necessarily the thing doing the walking. It is the thing that can hand Python a walker. In Python vocabulary, the walker is called an **iterator**.

1.3.3 Examples

```

1 iter([1, 2, 3]) # list is iterable
2 iter((1, 2, 3)) # tuple is iterable
3 iter("Python") # string is iterable
4 iter({"a": 1}) # dictionary is iterable over keys
5 iter({1, 2, 3}) # set is iterable
6 iter(range(3)) # range is iterable
7 iter(open("data.txt")) # file is iterable over lines
  
```

An integer is not iterable:

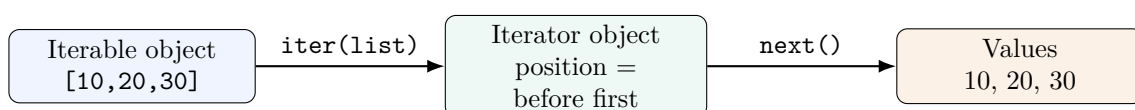
```

1 iter(42) # TypeError: 'int' object is not iterable
  
```

1.3.4 Why Integer Is Not Iterable

The number 42 is a single value, not a container, stream, or process. Python cannot ask, “What is the first element of 42? What is the second?” There is no natural current position and no sequence of next values.

1.3.5 Memory Diagram



Interview Insight

A list is iterable because it can create an iterator. A list is not consumed by one loop. You can loop over the same list again because each new `for` loop asks the list for a fresh iterator.

1.4 Iterator: The Object That Remembers the Walk

1.4.1 Problem

The iterable can be walked through, but something must remember the current position. If Python has already returned 10, the next request should return 20, not 10 again.

1.4.2 Inventing the Iterator Naturally

Imagine a museum audio guide. The museum is the iterable. The audio guide is the iterator. It remembers which exhibit you are standing near and plays the next description when you press “next.”

Iterator

An iterator is an object that returns one value at a time, remembers its current state, and raises `StopIteration` when there are no values left.

1.4.3 Manual Iteration

```
1 numbers = [10, 20, 30]
2 walker = iter(numbers)
3
4 print(next(walker))
5 print(next(walker))
6 print(next(walker))
```

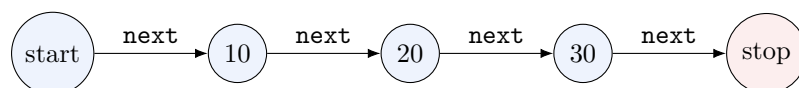
```
10
20
30
```

If we call `next(walker)` again, the iterator raises `StopIteration`.

1.4.4 Line-by-Line Output Explanation

Output	Explanation
10	The iterator was before the first item, so the first <code>next</code> returns 10.
20	The iterator saved its position after 10; the next request returns 20.
30	The iterator saved its position after 20; the next request returns 30.

1.4.5 Iterator State Diagram



1.5 The Iteration Protocol

1.5.1 Problem

Python must support many kinds of iterable objects without writing a special `for` loop for each type.

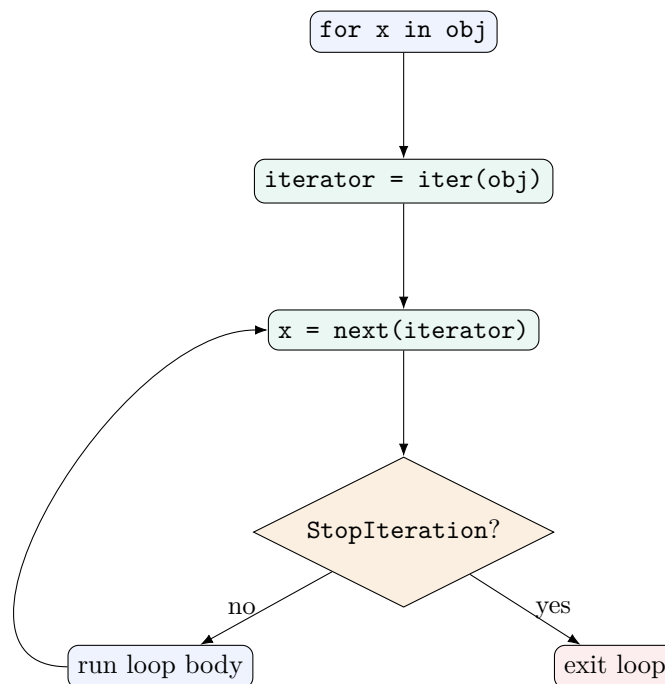
1.5.2 Need

The language defines a protocol: a small agreement between Python and objects.

The Complete Protocol

- `iter(obj)` asks an iterable for an iterator.
- Internally, `iter(obj)` usually calls `obj.__iter__()`.
- `next(iterator)` asks the iterator for the next value.
- Internally, `next(iterator)` calls `iterator.__next__()`.
- When values are exhausted, `__next__()` raises `StopIteration`.

1.5.3 Visualization



1.6 How a for Loop Works Internally

1.6.1 Problem

The code hides the protocol:

```
1 for x in numbers:
2     print(x)
```


1.6.2 Internal Translation

Conceptually, Python behaves like this:

```

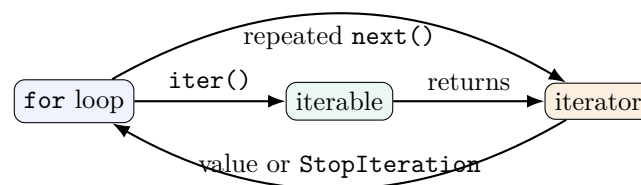
1 iterator = iter(numbers)
2
3 while True:
4     try:
5         x = next(iterator)
6         print(x)
7     except StopIteration:
8         break

```

1.6.3 Dry Run Table

Step	Operation	Result	Meaning
1	<code>iter(numbers)</code>	iterator created	Python gets a stateful walker.
2	<code>next(iterator)</code>	10	First value enters loop variable <code>x</code> .
3	loop body	prints 10	User code runs.
4	<code>next(iterator)</code>	20	Iterator advances.
5	loop body	prints 20	User code runs.
6	<code>next(iterator)</code>	30	Iterator advances.
7	loop body	prints 30	User code runs.
8	<code>next(iterator)</code>	raises <code>StopIteration</code>	Python exits loop silently.

1.6.4 Call Sequence Diagram



1.7 Custom Iterators

1.7.1 Why Build One?

You build a custom iterator when your object represents a custom walking process: counting, filtering, reading batches, walking a tree, producing alphabet letters, or exposing domain-specific records one at a time.

Iterator Class Pattern

A custom iterator usually stores state in attributes, returns `self` from `__iter__`, and advances state in `__next__`. When exhausted, it raises `StopIteration`.

1.7.2 Example 1: Counter Iterator

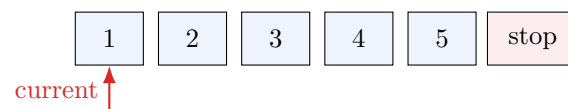
1. Problem

Produce numbers from `start` to `end`, one at a time, without returning a full list.

2. Explanation

The object must remember the current number. Every `next()` returns the current number and moves the current number forward.

3. Visualization



4. Algorithm Thinking

- Store `current` and `end`.
- If `current > end`, raise `StopIteration`.
- Otherwise save current value, increment `current`, and return saved value.

5. Pseudo Code

```
Counter(start, end):
    current = start
    end = end

next():
    if current > end:
        signal finished
    value = current
    current = current + 1
    return value
```

6. Python Code

```
1 class Counter:
2     def __init__(self, start, end):
3         self.current = start
4         self.end = end
5
6     def __iter__(self):
7         return self
8
9     def __next__(self):
10        if self.current > self.end:
11            raise StopIteration
12        value = self.current
13        self.current += 1
14        return value
```

7. Driver Code

```
1 for number in Counter(1, 5):
2     print(number)
```

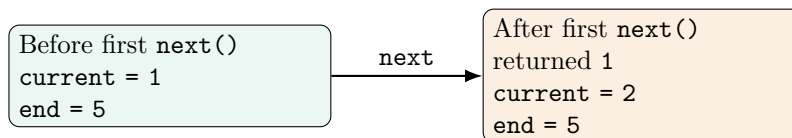
8. Console Output

```
1
2
3
4
5
```

9. Output Explanation

Each printed line is the value returned by one call to `__next__`. After 5, `current` becomes 6. The next call raises `StopIteration`, and the `for` loop exits.

10–13. Memory and Execution



Call	current before	returned	current after	Event
1	1	1	2	normal value
2	2	2	3	normal value
3	3	3	4	normal value
4	4	4	5	normal value
5	5	5	6	normal value
6	6	none	6	StopIteration

14. Complexity

Each `next()` call is $O(1)$. Total time for n produced numbers is $O(n)$. Extra memory is $O(1)$ because only `current` and `end` are stored.

15–17. Mistakes, Edge Cases, Interview

Common Mistakes

Forgetting to increment `current` creates an infinite iterator. Returning `None` instead of raising `StopIteration` makes `None` look like a real value and the loop may never stop.

Edge Cases

If `start > end`, the first `next()` raises `StopIteration`. This is correct: the iterator is empty.

Interview Question

Why does `__iter__` return `self`? Because this object is already the iterator; it does not need to create a separate walker.

1.7.3 Example 2: Countdown Iterator

```

1 class Countdown:
2     def __init__(self, start):
3         self.current = start
4
5     def __iter__(self):
6         return self
7
8     def __next__(self):
9         if self.current < 0:
10            raise StopIteration
11        value = self.current
12        self.current -= 1
13        return value
14
15 for value in Countdown(3):
16     print(value)

```

```

3
2
1
0

```

Call	Before	Output	Explanation
1	3	3	Return current, then decrement to 2.
2	2	2	Return current, then decrement to 1.
3	1	1	Return current, then decrement to 0.
4	0	0	Zero is still valid, then decrement to -1.
5	-1	none	Stop signal is raised.

1.7.4 Example 3: Even Numbers Iterator

```

1 class EvenNumbers:
2     def __init__(self, limit):
3         self.current = 0
4         self.limit = limit
5
6     def __iter__(self):
7         return self
8
9     def __next__(self):
10        if self.current > self.limit:
11            raise StopIteration

```

```
12     value = self.current
13     self.current += 2
14     return value
15
16 for number in EvenNumbers(8):
17     print(number)
```

```
0
2
4
6
8
```

1.7.5 Example 4: Odd Numbers Iterator

```
1 class OddNumbers:
2     def __init__(self, limit):
3         self.current = 1
4         self.limit = limit
5
6     def __iter__(self):
7         return self
8
9     def __next__(self):
10        if self.current > self.limit:
11            raise StopIteration
12        value = self.current
13        self.current += 2
14        return value
15
16 for number in OddNumbers(7):
17     print(number)
```

```
1
3
5
7
```

1.7.6 Example 5: Alphabet Iterator

```
1 class Alphabet:
2     def __init__(self, start="A", end="Z"):
3         self.current_code = ord(start)
4         self.end_code = ord(end)
5
6     def __iter__(self):
7         return self
8
9     def __next__(self):
10        if self.current_code > self.end_code:
11            raise StopIteration
12        letter = chr(self.current_code)
13        self.current_code += 1
14        return letter
15
```

```
16 for letter in Alphabet("A", "E"):
17     print(letter)
```

```
A
B
C
D
E
```

Reusable Example Template

For each iterator above, the same thinking applies: identify state, decide the next value, update state, stop correctly, test edge cases, and explain memory as stored attributes rather than stored output.

1.8 StopIteration

1.8.1 Why It Exists

A loop needs a clean end signal. Returning a special value such as `None` would be ambiguous because `None` might be a legitimate item in the sequence.

```
1 items = [1, None, 3]
```

If `None` meant “stop,” Python could not iterate over this list correctly. Therefore Python uses an exception: `StopIteration`.

1.8.2 What Happens If It Is Not Raised?

If `__next__` never raises `StopIteration`, the iterator appears infinite. That may be intentional for infinite streams, but it is a bug for finite sequences.

1.8.3 Why Users Rarely See It

A `for` loop catches `StopIteration` internally and exits silently. You usually see it only when manually calling `next()` too many times.

1.9 Limitations of Custom Iterators

Custom iterator classes work, but they can be verbose.

```
1 class SimpleSequence:
2     def __init__(self):
3         self.step = 0
4
5     def __iter__(self):
6         return self
7
8     def __next__(self):
9         if self.step == 0:
10             self.step += 1
11             return "prepare"
12         if self.step == 1:
13             self.step += 1
14             return "process"
```

```
15     if self.step == 2:
16         self.step += 1
17         return "finish"
18     raise StopIteration
```

Why Developers Wanted Something Simpler

Iterator classes require explicit state variables, explicit stopping logic, and scattered control flow. For many sequences, the human thought process is simple: “produce this, then this, then this.” Python needed a way to express that directly.

1.10 Introducing Generators

1.10.1 Problem

We want iterator behavior without writing a whole iterator class.

1.10.2 Curiosity

What if a function could return a value, pause exactly where it is, remember local variables, and continue from that exact line the next time we ask for a value?

That is the idea behind generators.

1.11 What Is a Generator?

Generator

A generator is an iterator created by a function containing `yield`. Calling the function does not run the body immediately. It returns a generator object. Each `next()` resumes the function until the next `yield`.

1.11.1 Generator Function and Generator Object

```
1 def simple_generator():
2     yield "prepare"
3     yield "process"
4     yield "finish"
5
6 g = simple_generator()
7 print(g)
```

- `simple_generator` is a generator function.
- `g` is a generator object.
- The body runs only when `next(g)` is called.

1.12 Yield: Return Versus Pause-and-Resume

1.12.1 Real-World Analogy

A normal `return` is like leaving a classroom and closing the course forever. A `yield` is like pausing the lesson, handing the student one result, keeping the board exactly as it is, and continuing from the next sentence when class resumes.

1.12.2 Return vs Yield Table

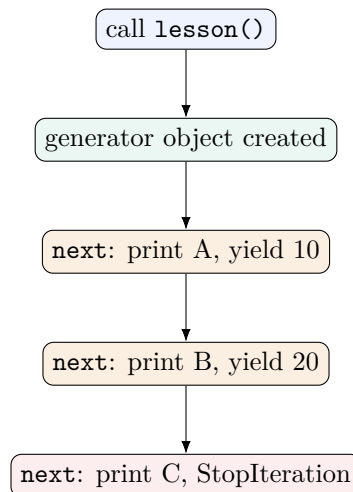
Feature	<code>return</code>	<code>yield</code>
Function type	Normal function	Generator function
Execution	Runs to completion when called	Body waits until <code>next()</code>
Value delivery	Sends one final result	Sends many values over time
State	Local frame is discarded	Local frame is suspended and preserved
Resume	Cannot resume after return	Resumes after each <code>yield</code>
End behavior	Function call is done	Exhaustion raises <code>StopIteration</code>

1.12.3 Execution Flow

```

1 def lesson():
2     print("A")
3     yield 10
4     print("B")
5     yield 20
6     print("C")

```



1.13 Generator Internal Execution

1.13.1 Complete Example

```

1 def count_to_three():
2     current = 1
3     yield current
4     current += 1

```



```

5     yield current
6     current += 1
7     yield current
8
9 g = count_to_three()
10 print(next(g))
11 print(next(g))
12 print(next(g))

```

1.13.2 Console Output

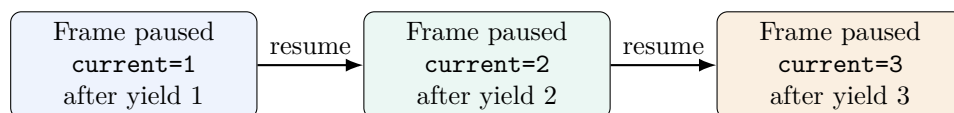
```

1
2
3

```

1.13.3 Memory After Every Yield

Call	Returned	Saved Local State	Instruction Pointer
1	1	current = 1	paused after first yield current
2	2	current = 2	paused after second yield current
3	3	current = 3	paused after third yield current
4	none	frame completes	raises StopIteration



How Yield Works Internally

A generator stores a suspended execution frame: local variables, the position of the next instruction, and enough execution context to continue later. It does not restart the function after each `next()`; it resumes from the suspension point.

1.14 Generator Expressions

1.14.1 Problem

List comprehensions are concise but build the entire list immediately.

```

1 squares = [x * x for x in range(1_000_000)]

```

If you only need values one at a time, storing all one million values may be wasteful.

1.14.2 Generator Expression

```

1 squares = (x * x for x in range(1_000_000))

```

The parentheses create a generator expression. It produces values lazily.

Feature	List Comprehension	Generator Expression
Syntax	[expr for x in data]	(expr for x in data)
Evaluation	Immediate	Lazy
Memory	Stores all results	Stores expression state
Indexing	Supported	Not supported
Reuse	Can loop many times	Usually consumed once
Best for	Small reusable data	Large or streaming data

1.14.3 Benchmark Discussion

A generator expression can use far less memory because it does not materialize all results. It may not always be faster: if you need to traverse the data many times, a list can be better because it stores computed results.

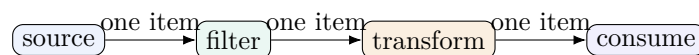
1.15 Lazy Evaluation

Lazy Evaluation

Lazy evaluation means delaying work until the result is actually requested. A generator does not compute the next value until `next()` asks for it.

1.15.1 Why It Is Powerful

- Huge files: read one line at a time.
- Infinite sequences: represent values that never end.
- Streaming APIs: process messages as they arrive.
- Database records: page through results instead of loading all rows.
- Machine learning: feed batches to a model gradually.
- Big data pipelines: transform data step by step without massive intermediate lists.



1.16 Infinite Generators

1.16.1 Safe Infinite Counter

```

1 def infinite_counter(start=0):
2     current = start
3     while True:
4         yield current
5         current += 1
6
7 counter = infinite_counter(10)
8 print(next(counter))
9 print(next(counter))
10 print(next(counter))

```

```
10
11
12
```

Safe Usage Rule

Never convert an infinite generator to a list. Use `next()`, `break`, slicing tools such as `itertools.islice`, or explicit limits.

1.16.2 Fibonacci Generator

```
1 def fibonacci():
2     a, b = 0, 1
3     while True:
4         yield a
5         a, b = b, a + b
6
7 fib = fibonacci()
8 for _ in range(7):
9     print(next(fib))
```

```
0
1
1
2
3
5
8
```

1.16.3 Prime Number Generator

```
1 def is_prime(number):
2     if number < 2:
3         return False
4     divisor = 2
5     while divisor * divisor <= number:
6         if number % divisor == 0:
7             return False
8         divisor += 1
9     return True
10
11 def primes():
12     number = 2
13     while True:
14         if is_prime(number):
15             yield number
16         number += 1
```

1.16.4 Streaming Sensor Analogy

A temperature sensor does not have tomorrow's readings stored today. It produces readings over time. A generator models this naturally: one reading now, another later, no need for a complete list.

1.17 Real-World Applications

1.17.1 Reading Huge Files

```
1 def read_lines(path):
2     with open(path, "r", encoding="utf-8") as file:
3         for line in file:
4             yield line.rstrip("\n")
```

Why generator? The file is processed line by line instead of loading every line into memory.

1.17.2 CSV Processing

```
1 import csv
2
3 def read_csv_rows(path):
4     with open(path, newline="", encoding="utf-8") as file:
5         reader = csv.DictReader(file)
6         for row in reader:
7             yield row
```

1.17.3 Log Stream Processor

```
1 def error_lines(lines):
2     for line in lines:
3         if "ERROR" in line:
4             yield line
```

1.17.4 Database Pagination

```
1 def paginated_records(fetch_page):
2     page_number = 1
3     while True:
4         page = fetch_page(page_number)
5         if not page:
6             break
7         for record in page:
8             yield record
9         page_number += 1
```

1.17.5 Batch Processor

```
1 def batches(items, size):
2     batch = []
3     for item in items:
4         batch.append(item)
5         if len(batch) == size:
6             yield batch
7             batch = []
8     if batch:
9         yield batch
```

1.17.6 Data Pipeline

```

1 def numbers(lines):
2     for line in lines:
3         yield int(line)
4
5 def only_even(values):
6     for value in values:
7         if value % 2 == 0:
8             yield value
9
10 def squared(values):
11     for value in values:
12         yield value * value
13
14 pipeline = squared(only_even(numbers(["1", "2", "3", "4"])))
15 print(list(pipeline))

```

```
[4, 16]
```

1.18 Iterator vs Generator

Aspect	Iterator	Generator
Definition	Object implementing <code>__iter__</code> and <code>__next__</code>	Iterator created by generator function or expression
Implementation	Class with explicit state	Function with <code>yield</code>
Complexity	More boilerplate	Less boilerplate
Memory	Usually stores only state	Stores suspended frame and locals
Speed	Can be highly customized	Usually excellent for simple sequences
Ease	Verbose for simple cases	Very readable for sequential logic
Flexibility	Full class design possible	Best for value streams
Reuse	Depends on design	Generator object is consumed once
Use cases	Custom containers, complex APIs	Streams, pipelines, lazy sequences
Interview angle	Shows protocol mastery	Shows lazy execution mastery

1.19 A Universal Code Example Checklist

For every serious iterator or generator example, use this checklist:

1. Problem
2. Explanation
3. Visualization
4. Algorithm thinking
5. Pseudo code
6. Python code
7. Driver code
8. Console output

- | | |
|--------------------------------|-------------------------|
| 9. Output explanation | 14. Complexity |
| 10. Memory before | 15. Common mistakes |
| 11. Memory after | 16. Edge cases |
| 12. Internal execution diagram | 17. Interview questions |
| 13. Dry run table | |

1.20 Fifty Common Beginner Mistakes

No.	Mistake	Why It Happens	Expert Fix
1	Calling <code>next()</code> too many times	Forgetting exhaustion exists	Expect <code>StopIteration</code> or use <code>for</code> .
2	Confusing iterable and iterator	Both appear in loops	Iterable gives iterator; iterator gives values.
3	Forgetting <code>iter()</code> before manual <code>next()</code>	Trying <code>next([1,2])</code>	Use <code>it = iter([1,2])</code> .
4	Expecting generators to restart	Thinking function call and generator object are same	Create a new generator object.
5	Using <code>return</code> instead of <code>yield</code>	Normal function habit	Use <code>yield</code> for streamed values.
6	Trying indexing on generator	Treating it like a list	Use <code>next()</code> or convert only if small.
7	Converting huge generator to list	Debugging convenience	Inspect first few values only.
8	Infinite generator inside <code>list()</code>	Forgetting no end exists	Use limits or <code>islice</code> .
9	Not raising <code>StopIteration</code>	Missing end condition	Add explicit exhaustion branch.
10	Raising <code>StopIteration</code> inside generator body manually	Mixing old patterns	Prefer <code>return</code> to end generator.
11	Returning <code>None</code> to stop iterator	Sentinel confusion	Raise <code>StopIteration</code> .
12	Forgetting state update	Infinite repeated value	Update state before returning or yielding.
13	Updating state too early	Skips first value	Dry run first call carefully.
14	Off-by-one limit	Inclusive/exclusive confusion	Define boundary before coding.
15	Reusing consumed iterator	Iterator state is exhausted	Recreate iterator.
16	Assuming sets have meaningful order	Set iteration order is not semantic	Do not rely on set order.
17	Mutating collection during iteration	State becomes unstable	Iterate over copy or collect changes.
18	Forgetting files are iterators over lines	Calling <code>readlines()</code> unnecessarily	Loop directly over file object.
19	Holding all pipeline output	Losing lazy benefit	Stream through pipeline.

20	Nesting generators without clarity	Hard debugging	Name pipeline stages.
21	Using generator when random access is needed	Wrong data shape	Use list when indexing is required.
22	Using list when one pass is enough	Habit from beginner code	Use generator for one-pass streams.
23	Forgetting <code>__iter__</code>	Object cannot be used in <code>for</code>	Implement protocol fully.
24	Returning new iterator from iterator <code>__iter__</code> incorrectly	Confusing container and iterator roles	Return <code>self</code> for iterator objects.
25	Making iterable and iterator same when reuse is needed	Consumed object surprises users	Use separate iterator class.
26	Swallowing <code>StopIteration</code> wrongly	Broad exception handling	Catch only where appropriate.
27	Thinking <code>yield</code> means thread pause	Concurrency confusion	It is cooperative suspension, not parallelism.
28	Expecting generator locals to reset	Misunderstanding frame preservation	Locals persist between yields.
29	Forgetting generator body does not run at call	Surprised prints do not appear	Call <code>next()</code> to start.
30	Putting expensive work before first yield	Delayed first result	Move setup carefully or document cost.
31	Not closing files inside generators	Resource leak risk	Use <code>with</code> inside generator.
32	Yielding mutable batch then reusing same list	All batches change together	Yield a copy or create a new list.
33	Catching all exceptions as stop	Hides bugs	Use clear end conditions.
34	Comparing generator to list directly	Different types	Convert intentionally or compare produced values.
35	Printing generator object expecting contents	Object representation appears	Use <code>list(g)</code> for small demos.
36	Forgetting dictionary iteration gives keys	Expecting key-value pairs	Use <code>items()</code> .
37	Assuming <code>range</code> stores all numbers	Old Python 2 memory model	In modern Python, <code>range</code> is lazy-like.
38	Overengineering simple generator as class	Boilerplate habit	Prefer <code>yield</code> for simple streams.
39	Underengineering complex iterator as generator	Too much hidden state	Use class when state needs methods.
40	Not testing empty input	Happy path only	Dry run zero elements.
41	Not testing single element	Boundary blind spot	Include one-item test.
42	Misusing <code>break</code> in generator pipeline	Stops too early	Know whether to skip or end.
43	Using <code>return value</code> expecting yielded value	Return value becomes <code>StopIteration</code> value	Use <code>yield value</code> .

44	Forgetting generator expressions use parentheses	Syntax confusion	Use <code>(...)</code> not <code>[...]</code> .
45	Exhausting generator during debugging	<code>list(g)</code> consumes it	Recreate generator after inspection.
46	Sharing one iterator across consumers	Consumers steal values	Give each consumer its own iterator.
47	Assuming <code>iter(iterator)</code> resets it	It returns same iterator	Build a new iterator to restart.
48	Ignoring memory of collected results	Generator not enough if you append all results	Avoid accumulating unless needed.
49	Forgetting line endings in file iteration	Newline surprises	Use <code>rstrip("\n")</code> intentionally.
50	Memorizing protocol without intuition	Syntax-first learning	Think current, next, end.

1.21 Interview Questions and Model Answers

Core Interview Set

1. **Why are generators memory efficient?** They produce one value at a time and store only execution state instead of a full result list.
2. **How does yield work internally?** It returns a value to the caller while suspending the generator frame, including local variables and instruction position.
3. **Difference between iterable and iterator?** An iterable can provide an iterator; an iterator produces values and remembers progress.
4. **Difference between iterator and generator?** A generator is a convenient kind of iterator created with `yield`; an iterator is the general protocol concept.
5. **Can a generator be reused?** A generator object is consumed once. Call the generator function again to get a fresh object.
6. **What happens after StopIteration?** The iterator is exhausted; future `next()` calls generally continue raising `StopIteration`.
7. **Why does next() raise an exception?** Because every normal value, including `None`, could be legitimate; an exception is an unambiguous end signal.
8. **How does a for loop work internally?** It calls `iter()`, repeatedly calls `next()`, catches `StopIteration`, and exits.

1.22 Practice Questions

Easy

1. Explain why 42 is not iterable.
2. Manually iterate over "abc" using `iter()` and `next()`.
3. Predict what happens after calling `next()` four times on `iter([1,2,3])`.

4. Write a generator that yields "red", "green", and "blue".

Medium

1. Build a `CustomRange(start, stop, step)` iterator.
2. Write a generator that yields the first `n` Fibonacci numbers.
3. Write a generator expression for squares of even numbers from 1 to 100.
4. Debug an iterator that never stops because it forgets to update state.

Hard

1. Implement a binary tree iterator that yields values in-order.
2. Build a lazy log-processing pipeline: read lines, filter errors, parse timestamps, batch results.
3. Write a prime generator and optimize divisibility checks.
4. Design an iterable class that returns a fresh iterator every time it is looped over.

Output Prediction and Dry Run

```

1 def mystery():
2     print("start")
3     yield 1
4     print("middle")
5     yield 2
6     print("end")
7
8 g = mystery()
9 print("A")
10 print(next(g))
11 print("B")
12 print(next(g))

```

Predict every output line and explain why the word `start` does not print when `g = mystery()` executes.

1.23 Mini Projects

Project	Goal	Key Concept
Infinite Counter	Yield numbers forever from a start value	Infinite generator safety
Custom Range	Recreate core behavior of <code>range</code>	Iterator protocol
Fibonacci Generator	Produce Fibonacci lazily	Saved generator state
Prime Number Generator	Yield primes indefinitely	Lazy filtering
File Reader	Yield one line at a time	Memory efficiency
CSV Reader	Yield dictionaries row by row	Streaming structured data
Log Stream Processor	Filter and transform log lines	Generator pipeline

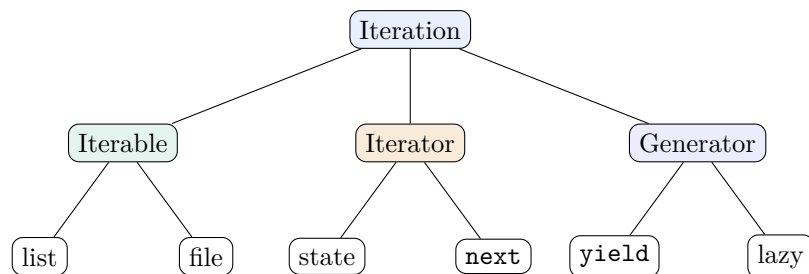
Pagination	Generator	Fetch page after page	External data streaming
Batch Processor		Group incoming items	Stateful generator
Data Pipeline		Compose multiple lazy stages	End-to-end lazy evaluation

1.24 Summary

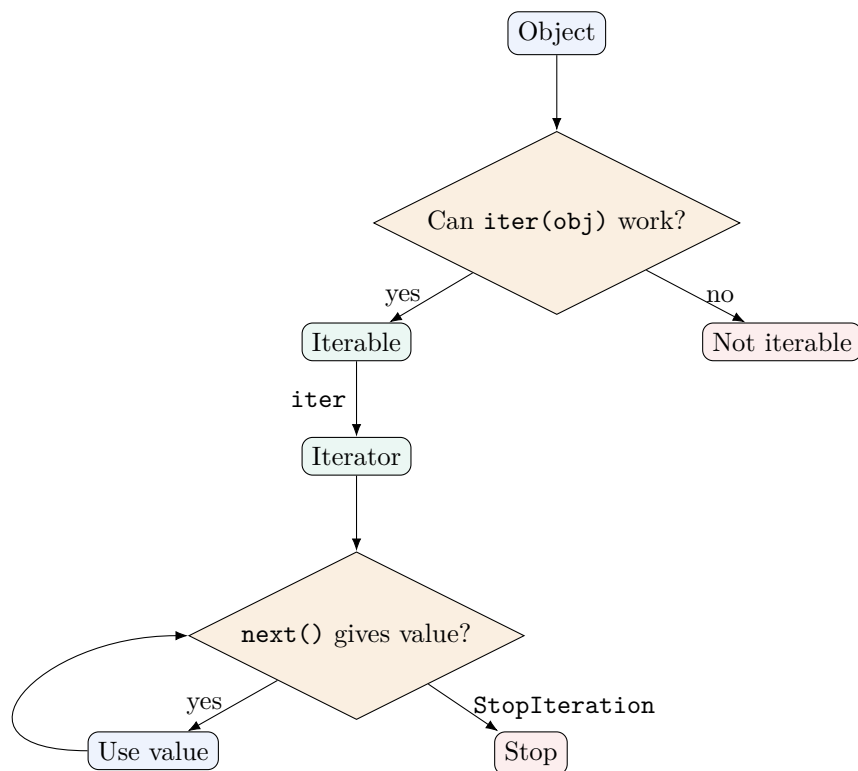
1.24.1 Cheat Sheet

Term	Meaning
Iteration	Walking through values one at a time.
Iterable	Object that can provide an iterator.
Iterator	Object that remembers position and returns next values.
<code>iter()</code>	Requests an iterator from an iterable.
<code>next()</code>	Requests the next value from an iterator.
<code>StopIteration</code>	Exception that signals no more values.
<code>__iter__</code>	Method called by <code>iter()</code> .
<code>__next__</code>	Method called by <code>next()</code> .
Generator function	Function containing <code>yield</code> .
Generator object	Iterator returned by calling a generator function.
<code>yield</code>	Produces a value and suspends execution state.
Generator expression	Lazy comprehension-like generator.
Lazy evaluation	Compute only when requested.

1.24.2 Mind Map



1.24.3 Flow Chart



1.24.4 Revision Notes

- Think **current**, **next**, **end**. This removes most confusion.
- An **iterable** is the source; an **iterator** is the stateful walker.
- A **for** loop is not magic: it calls **iter()**, calls **next()**, and catches **StopIteration**.
- A generator is a compact iterator whose local frame is suspended at **yield**.
- Lazy evaluation saves memory and enables streams, files, pipelines, and infinite sequences.
- Use a class iterator for rich custom objects; use a generator for readable value streams.

1.24.5 Memory Tricks

- **Iterable**: “I can be walked.”
- **Iterator**: “I am the walker.”
- **Generator**: “I am a paused function acting as a walker.”
- **StopIteration**: “The walker says the path is finished.”
- **Lazy**: “Do not cook the whole meal; prepare each bite when requested.”